



Programação WEB Avançada

Universidade de Vila Velha

Comprometida com a excelência no ensino, pesquisa e inovação.

✉ wanderson.santana@uvv.br

2

Flask: uma primeira aplicação

Resumo

Esta unidade apresenta uma introdução prática e estruturada ao desenvolvimento de aplicações web com Flask, utilizando boas práticas de modularização e configuração. Iniciamos com a preparação do ambiente de desenvolvimento, incluindo o uso de ambientes virtuais e a instalação de dependências. Em seguida, detalhamos a criação da aplicação por meio do padrão *factory*, o uso de *blueprints* para organização das rotas, e a separação de responsabilidades em subpacotes. Abordamos também o gerenciamento de diferentes ambientes de execução (*development*, *production*) utilizando variáveis de ambiente e arquivos de configuração. Por fim, explicamos como executar a aplicação localmente, destacando comandos específicos para diferentes sistemas operacionais e o papel das variáveis `FLASK_APP`, `FLASK_ENV` e `FLASK_DEBUG`. Este material serve como base sólida para projetos Flask escaláveis, reutilizáveis e compatíveis com testes automatizados.

“A ordem é a forma mais bela da complexidade.”

José Ortega y Gasset, *Meditaciones del Quijote* (1914)



Desvendando o Flask

Estamos prestes a iniciar uma jornada para aprender a **criar aplicações web** utilizando **Python** e o **microframework Flask**. Nesta primeira etapa, vamos aprender como **configurar** um projeto **Flask** do zero. Ao final, você terá um **aplicativo web** simples rodando **localmente** em seu **computador**!

Se você estiver utilizando um **sistema operacional** baseado em **Unix**, como Linux ou macOS, é bem provável que o **Python** já esteja instalado. No entanto, usuários do Windows devem estar atentos: esse sistema não vem com o Python instalado por padrão. Nesse caso, recomendamos que você faça a instalação manualmente. Acesse o site oficial do Python¹ e baixe o instalador adequado para o seu sistema.

Para garantir que sua instalação do **Python** está funcionando corretamente, abra uma janela de terminal e digite `python3` (em sistemas Linux ou macOS) ou `py` (no Windows). Caso nenhum desses comandos funcione, tente simplesmente `python`. Se tudo estiver certo, você deverá ver algo semelhante ao seguinte:

```
Python 3.12.3 (main, Jan 17 2025, 18:03:48) [GCC 13.3.0] on ↵  
↳ linux  
Type "help", "copyright", "credits" or "license" for more ↵  
↳ information.  
>>>
```

Neste ponto, o interpretador interativo do **Python** está em **execução**, pronto para receber comandos. Você pode digitar instruções diretamente para testá-las.

Para **sair** do **interpretador interativo**, você pode:

- Digitar `exit()` e pressionar Enter;
- Pressionar Ctrl-D (em Linux/macOS);
- Pressionar Ctrl-Z seguido de Enter (em Windows).

¹Disponível em: <https://www.python.org/downloads/windows/>

O próximo passo será instalar o **Flask**. No entanto, antes de prosseguirmos, é importante abordar algumas **boas práticas** relacionadas à **instalação de pacotes** em **Python**.

Pacotes adicionais, como o **Flask**, estão disponíveis em um repositório público chamado PyPI — sigla para **Python Package Index**. Esse **repositório** contém milhares de **bibliotecas** que podem ser facilmente instaladas com a **ferramenta pip**, que já vem incluída na maioria das distribuições modernas do Python.

Para instalar um pacote de forma **global** (ou seja, disponível para **todo o sistema**), você pode executar:

```
pip install <nome-do-pacote>
```

No entanto, realizar **instalações globais** pode acarretar **problemas** importantes:

- Se o Python foi instalado **globalmente** (por exemplo, por um **administrador do sistema**), é provável que você **não** tenha permissão para instalar pacotes sem o uso de **sudo** (em Linux/macOS) ou sem executar o terminal como administrador (no Windows).
- Além disso, **pacotes instalados globalmente** ficam disponíveis para **todos os scripts** Python em sua máquina, o que pode causar **conflitos** entre projetos que dependem de versões diferentes da **mesma biblioteca**.
- Em sistemas baseados em **Unix** (como Linux e macOS), há ainda o risco de **corromper dependências críticas** do próprio **sistema operacional**, comprometendo sua **estabilidade**.

Por essas razões, a prática recomendada é utilizar **ambientes virtuais** para **cada projeto**. Eles proporcionam um **ambiente isolado**, onde pacotes podem ser instalados e gerenciados sem afetar o restante do sistema.

Uma discussão mais detalhada sobre o uso adequado do **pip** pode ser encontrada no **Apêndice B**, que apresenta uma visão geral do **gerenciamento de pacotes em ambientes virtuais Python**. Essa discussão é baseada nas **Propostas de Melhoria do Python (PEPs) 405, 668 e 704**, além das **diretrizes da comunidade** presentes no **guia oficial** de **empacotamento** da linguagem.

Ambientes Virtuais: a solução recomendada

Para contornar os problemas da instalação global de pacotes e manter seu ambiente de desenvolvimento mais organizado e seguro, o `Python` oferece uma ferramenta chamada `venv`, que permite criar **ambientes virtuais**.

Um ambiente virtual é uma **instalação isolada** do **interpretador Python**, onde você pode instalar **pacotes** sem afetar a **instalação global do sistema** — nem **interferir** em outros projetos. Isso significa que:

- cada **projeto** pode ter suas **próprias versões** de pacotes;
- você evita **conflitos** entre dependências;
- não precisa de **permissões administrativas** para **instalar pacotes**;
- seu ambiente se torna mais fácil de **manter** e **replicar** (com arquivos como `pyproject.toml`).

Portanto, a prática recomendada é: **sempre crie e utilize um ambiente virtual para cada novo projeto em Python.**

Criando e ativando um ambiente virtual

Vamos começar criando um **diretório** para o projeto. Por exemplo:

```
$ mkdir UVV
$ cd UVV
```

Agora, crie o ambiente virtual:

```
$ python3 -m venv venv
```

Esse comando instrui o Python a executar o **módulo `venv`**, criando um **ambiente virtual** chamado `venv` dentro do **diretório** do projeto. Se preferir, você pode dar outro nome ao ambiente, como `.env`, `env` ou `meu_ambiente`, por exemplo.

NOTA

Em alguns sistemas operacionais, o comando pode ser `python` ou `py` em vez de `python3`, dependendo da configuração local.

Após a criação, será gerado um **diretório `venv`** contendo **todos os arquivos** e **binários** do novo **ambiente virtual**.

Ativando o ambiente virtual

Para ativar o ambiente virtual:

- Em **Linux/macOS**:

```
$ source venv/bin/activate  
(venv) $
```

- Em **Windows CMD**:

```
C:\UVV> venv\Scripts\activate.bat  
(venv) C:\UVV>
```

- Em **Windows (PowerShell)**:

```
$ Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -<>  
    <>Scope CurrentUser  
$ venv\Scripts\activate  
(venv) $
```

Para o PowerShell, o primeiro comando no Windows **ajusta a política de execução** para **permitir scripts locais assinados** — útil caso você não tenha **permissões administrativas**. Ele altera a **configuração** apenas para o **usuário atual**.

Ao ativar um ambiente virtual, a localização do seu interpretador Python é adicionada temporariamente à variável PATH da sessão. Isso garante que, ao digitar python, o sistema use o interpretador do ambiente virtual.

Além disso, o prompt do terminal será modificado para incluir o nome do ambiente ativo (como (venv)), indicando que você está dentro do ambiente.

NOTA

A ativação só vale para a sessão atual do terminal. Se você abrir uma nova janela, será necessário ativar o ambiente novamente. É possível ter múltiplas sessões com diferentes ambientes virtuais ativos simultaneamente.

Instalando o Flask no ambiente virtual

Com o ambiente virtual ativado, agora podemos instalar o Flask:

```
(venv) $ pip install flask
```

Para verificar se o Flask foi instalado corretamente:

```
(venv) $ python
>>> import flask
>>> _
```

Se o comando acima não gerar erros, o Flask foi instalado com sucesso!

NOTA

O comando de instalação sem especificação de versão instalará a versão mais recente disponível. Para este curso, utilizaremos o [Flask versão 3.x](#) ou superior — e o comando acima instalará automaticamente a versão adequada, desde que esteja disponível no PyPI.

O que é o Flask?

O Flask é considerado um **micro-framework** para aplicações web[1], pois oferece apenas os componentes essenciais para iniciar um projeto — como roteamento de URLs e suporte a templates. Isso não significa que ele seja limitado: sua arquitetura modular e extensível permite adicionar apenas os componentes de que você realmente precisa.

Ao contrário de frameworks mais robustos, que vêm com muitas ferramentas integradas (como autenticação, ORM, formulários etc.), o Flask permite que você escolha quais extensões utilizar — ou até mesmo crie as suas próprias.

Principais dependências do Flask

O Flask possui três dependências principais:

- **Werkzeug** — fornece o sistema de roteamento, suporte a WSGI e ferramentas de depuração;
- **Jinja2** — motor de templates, que permite separar lógica e apresentação em HTML;
- **Click** — framework para criar interfaces de linha de comando, usado pelo Flask CLI.

Essas três bibliotecas foram desenvolvidas por **Armin Ronacher**, o criador do Flask, e são mantidas ativamente pela comunidade.

NOTA

O Flask não oferece suporte nativo para algumas funcionalidades comuns em aplicações web, como acesso a banco de dados, autenticação ou validação de formulários. No entanto, há diversas extensões para isso — como Flask-SQLAlchemy, Flask-Login e Flask-WTF — que veremos mais adiante no curso.

A construção de um aplicativo web em Python com o Flask exige uma abordagem de desenvolvimento orientada à organização por **pacotes**, promovendo maior modularidade e reutilização de código.

Outro conceito comum no ecossistema Python, especialmente em **aplicações web**, é o padrão chamado **Application Factory** — ou simplesmente **Factory**. Esse padrão favorece a **criação dinâmica** de **instâncias da aplicação**, com configurações **flexíveis** e **adaptáveis** a diferentes contextos (como **desenvolvimento**, **testes** e **produção**).

Exploraremos esse conceito com mais profundidade nas próximas seções.

Inicializando uma aplicação

Todo **aplicativo Flask** precisa criar uma **instância principal da aplicação**. Essa **instância** funciona como o **núcleo da aplicação web**: é para ela que o **servidor web** envia as requisições dos clientes, por meio do **protocolo WSGI** (**Web Server Gateway Interface**).

A **instância** é criada a partir da **classe Flask**, da seguinte forma:

```
1 from flask import Flask
2 app = Flask(__name__)
```

O argumento obrigatório para o construtor `Flask()` é o **nome do módulo** ou **pacote principal**. O valor `__name__`, uma variável especial do Python, indica o **nome do módulo atual** e é usado pelo Flask para **localizar arquivos estáticos**, **templates** e outras **funcionalidades internas**. Portanto, para a maioria das aplicações, passar `__name__` é suficiente.

Uma abordagem profissional desde o início

Poderíamos, neste ponto, criar uma **aplicação** básica em um **único arquivo Python** (como `hello.py`). No entanto, isso não reflete a forma como **aplicações Flask** reais são estruturadas.

Em aplicações profissionais, buscamos manter o **projeto modularizado**, **escalável** e **testável** desde o início. Por isso, ao invés de criar a **instância do aplicativo** diretamente em um **arquivo**, vamos aplicar uma abordagem mais robusta: o **padrão de projeto** chamado **Application Factory**.

Por que usar uma Application Factory?

Ao utilizar uma função que *fabrica* (ou constrói) instâncias do aplicativo Flask sob demanda, ganhamos os seguintes benefícios:

- Podemos criar múltiplas instâncias da aplicação com configurações diferentes (útil para ambientes de desenvolvimento, testes e produção);
- Podemos inicializar extensões (como banco de dados, autenticação, etc.) somente quando necessário;
- Melhor suporte a testes automatizados;
- Melhor organização do código.

A função factory será colocada dentro de um pacote app, mais especificamente no arquivo `__init__.py`. Em Python, qualquer diretório contendo um arquivo `__init__.py` é tratado como um pacote, e esse arquivo define o comportamento de importação do pacote.

Quando importamos esse pacote, o código dentro do `__init__.py` é executado – é aí que definiremos a criação da aplicação Flask.

Esse ponto é fundamental para a estrutura da nossa aplicação.

Estrutura inicial do projeto

Abaixo, temos a estrutura inicial do nosso projeto. Ela já adota o padrão de modularização com base em pacotes, e inclui a criação de um ambiente virtual:

Da Figura (2.1) vê-se claramente que para organizar o desenvolvimento da aplicação Flask de maneira clara e escalável, adotamos uma estrutura de diretórios que separa responsabilidades e facilita a manutenção do código.

A seguir, vamos detalhar os principais arquivos e pastas presentes no projeto:

- **app.py**: Este é o ponto de entrada da aplicação. Normalmente, ele importa a função `factory create_app` do pacote app e inicializa a instância do Flask, além de configurar o servidor para rodar o aplicativo.

```

UVV/
|
+-- app.py
+-- config.py
+-- app/
|   +-- __init__.py           % Fábrica da aplicação Flask
|   +-- main/
|       +-- __init__.py       % Definição do blueprint
|       +-- routes.py         % Definição das rotas
|
+-- venv/                     % Ambiente virtual Python

```

Figura 2.1: Estrutura de diretórios do projeto Flask

- **config.py**: Contém as classes de configuração para diferentes ambientes (desenvolvimento, produção, testes). Aqui definimos parâmetros essenciais como a chave secreta, configuração do banco de dados, opções de debug, entre outros.
- **app/**: Diretório que representa o pacote principal da aplicação Flask. Nele estão os módulos e subpacotes que implementam a lógica da aplicação.
 - **__init__.py**: Arquivo responsável por implementar a função `factory create_app` e registrar os blueprints da aplicação.
 - **main/**: Subpacote que concentra a funcionalidade principal da aplicação.
 - * **__init__.py**: Define o blueprint main da aplicação.
 - * **routes.py**: Contém a definição das rotas (URLs) e as funções associadas para tratar as requisições HTTP.
- **venv/**: Ambiente virtual Python, onde ficam isoladas as dependências do projeto, incluindo o Flask e outras bibliotecas necessárias.

Essa organização modular permite desenvolver, testar e manter a aplicação de maneira eficiente, além de facilitar a escalabilidade conforme o projeto cresce[2].

Nas próximas seções, exploraremos com mais detalhes cada um desses componentes, começando pela função `create_app()`, que será implementada dentro de `app/__init__.py`. Este

será o ponto de entrada para construir a instância do Flask com todas as configurações necessárias.

Começemos, portanto, a construir essa função factory passo a passo.

Implementando a função factory do Flask

O script a seguir (`app/__init__.py`) cria o objeto do aplicativo como uma instância da classe `Flask`, que utiliza o local do módulo passado (neste caso, a pasta `app`) como ponto de partida para carregar recursos associados, como arquivos de templates, métodos e atributos, que exploraremos mais adiante. Na maioria dos casos, passar `__name__` como argumento garante que o `Flask` seja configurado corretamente.

Em projetos Python, a presença do arquivo `__init__.py` dentro de uma pasta indica que essa pasta deve ser tratada como um pacote Python. Isso permite que o interpretador reconheça o diretório como um módulo, possibilitando a importação de seus submódulos e facilitando a organização do código em múltiplos arquivos e subpastas.

No contexto de aplicações Flask, essa prática é fundamental para a criação e estruturação de blueprints, configuração de pacotes e para garantir que os componentes da aplicação sejam corretamente carregados e acessíveis. Além disso, o arquivo `__init__.py` é normalmente o local onde inicializamos objetos importantes, como a instância do blueprint, a conexão com bancos de dados, ou outras configurações essenciais para o funcionamento do pacote.

A função factory `create_app` é uma prática recomendada para aplicações que crescem em complexidade, especialmente quando precisamos aplicar configurações diferentes para múltiplos ambientes, como desenvolvimento, produção e testes. No exemplo abaixo, a função ajusta a configuração do app com base no parâmetro `config_name`, carregando o perfil adequado definido no arquivo `config.py` (que abordaremos a frente).

Esse formato facilita a inclusão automática de configurações essenciais, como o modo de debug, a chave secreta, parâmetros do banco de dados, entre outros.

O comando `from .main import main as main_blueprint` importa o blueprint principal da aplicação, enquanto `app.register_blueprint(main_blueprint)` registra esse blue-

`print` para gerenciar rotas e lógica específicas. Já falamos sobre `blueprints` em detalhes na **Unidade 01**.

O retorno do objeto `app` ao final da função `create_app` é fundamental, pois permite que a instância configurada da aplicação Flask seja utilizada em outros módulos, scripts ou mesmo em testes automatizados. Dessa forma, a `factory` não apenas cria a aplicação, mas também disponibiliza uma instância pronta para ser executada ou estendida conforme necessário. Essa abordagem modular facilita a manutenção, a escalabilidade e a testabilidade do seu projeto Flask.

Código: `app/__init__.py`

```
1 from flask import Flask
2
3 def create_app(config_name='development'):
4     """
5     Cria e configura uma instancia do aplicativo Flask.
6     """
7     app = Flask(__name__)
8
9     if config_name == 'development':
10         app.config.from_object('config.DevelopmentConfig')
11     elif config_name == 'production':
12         app.config.from_object('config.ProductionConfig')
13     else:
14         app.config.from_object('config.DefaultConfig')
15
16     # Define o modo debug conforme a configuracao
17     app.debug = app.config.get('DEBUG', False)
18
19     from .main import main as main_blueprint
20     app.register_blueprint(main_blueprint)
21
22     return app
```

A seguir, vamos trabalhar no desenvolvimento do subpacote que concentra a funcionalidade principal da aplicação — o pacote `main`.

app/main/___init___py

O arquivo `__init__.py` dentro da pasta `main` é responsável por criar o **blueprint** principal da aplicação.

Um *blueprint* é uma forma de organizar um conjunto de rotas e funcionalidades relacionadas dentro da aplicação Flask, facilitando a modularização e reaproveitamento de código.

Código: app/main/__init__.py

```
1 from flask import Blueprint
2
3 # Criacao do blueprint 'main'
4 main = Blueprint('main', __name__)
5
6 # Importa as rotas associadas a esse blueprint
7 from . import routes
```

Ao escrever o comando `main = Blueprint('main', __name__)`, estamos criando uma variável chamada `main` que representa um blueprint no Flask — um componente modular da aplicação web. Essa variável é uma instância da classe `Blueprint`, utilizada para registrar rotas, handlers e outras funcionalidades específicas desse módulo. O segundo argumento, `__name__`, informa ao Flask qual é o módulo atual, permitindo que o framework localize corretamente recursos associados, como templates e arquivos estáticos relacionados a esse blueprint.

Em seguida, importamos o módulo `routes` para registrar as rotas definidas no blueprint.

app/main/routes.py

O arquivo `routes.py` define as rotas que pertencem ao blueprint `main`. Cada rota é associada a uma função que será executada quando a URL correspondente for acessada.

No exemplo abaixo, definimos duas rotas: a raiz do site `("/")` e `"/index"`. Ambas estão vinculadas à função `index`, que retorna uma mensagem simples.

Dentro da função, usamos o objeto `current_app` para acessar a configuração corrente da aplicação e recuperar a variável `CONFIG_NAME`, demonstrando como personalizar o comportamento conforme o ambiente em que a aplicação está rodando (desenvolvimento, produção, etc.).

Os decorators `@main.route('/')` e `@main.route('/index')` vinculam essas URLs às funções Python correspondentes dentro do blueprint `main`, garantindo que as rotas estejam registradas no escopo correto da aplicação modularizada.

Código: app/main/routes.py

```
1 from flask import current_app
2 from . import main
3
4 @main.route('/')
5 @main.route('/index')
6 def index():
7     ambiente = current_app.config.get("CONFIG_NAME", "desconhecido")
8     return f'Ola, mundo! Ambiente atual: {ambiente}."
```

Essa organização modular por meio de **blueprints** facilita a **manutenção** e **expansão** da **aplicação**, agrupando **funcionalidades** em **subpacotes independentes**.

app.py

O **arquivo app.py** é o **ponto de entrada** da **aplicação Flask**. Nele, o **ambiente de configuração** é determinado pela **variável de ambiente FLASK_CONFIG** ou, se esta não estiver definida, por **FLASK_ENV**. Caso nenhuma delas esteja presente, o **valor padrão 'default'** é utilizado.

Com base nessa **configuração**, a **função create_app** (implementada anteriormente) é **chamada** para **criar** a **instância do aplicativo** **configurada adequadamente**.

Código: app.py

```
1 import os
2 from app import create_app
3
4 config_name = os.getenv('FLASK_CONFIG') or os.getenv('FLASK_ENV') or 'default'
5 app = create_app(config_name)
6
7 print(f"[DEBUG] app.debug = {app.debug}")
8 print(f"[DEBUG] config_name = {config_name}")
9
10 if __name__ == '__main__':
11     app.run()
```

As mensagens de debug exibidas ajudam a confirmar qual configuração está em uso e se o modo debug está ativado.

Por fim, o aplicativo é executado apenas se o script for executado diretamente, graças à condicional `if __name__ == '__main__':`, garantindo que o servidor Flask seja iniciado apenas nesse contexto.

config.py

O arquivo `config.py` centraliza as configurações da aplicação para diferentes ambientes, utilizando classes para representar cada perfil.

A classe `DefaultConfig` define as configurações base, como desativar o modo debug e testes, e atribui uma chave secreta padrão (que idealmente deve ser substituída por uma variável de ambiente segura).

As subclasses `DevelopmentConfig` e `ProductionConfig` herdam de `DefaultConfig`, sobrescrevendo ou adicionando parâmetros específicos para os respectivos ambientes, como ativar o modo debug em desenvolvimento.

Código: config.py

```
1 import os
2
3 class DefaultConfig:
4     DEBUG = False
5     TESTING = False
6     SECRET_KEY = os.getenv('SECRET_KEY', 'chave-padrao-insegura')
7
8 class DevelopmentConfig(DefaultConfig):
9     DEBUG = True
10    TESTING = True
11    ENV = 'development'
12    FLASK_DEBUG = 1
13
14 class ProductionConfig(DefaultConfig):
15     DEBUG = False
16     ENV = 'production'
```

Esse padrão orientado a classes facilita a manutenção das configurações e a alternância entre diferentes perfis da aplicação, sem a necessidade de alterar o código principal.

2.1 Executando a aplicação localmente

Com todos os arquivos implementados, já é possível executar a aplicação Flask localmente. Para isso, siga os passos descritos abaixo com o ambiente virtual ativado.

1. Definindo variáveis de ambiente

Antes de iniciar o servidor Flask, defina as variáveis de ambiente necessárias para informar ao framework qual arquivo carregar e qual configuração aplicar:

```
$ export FLASK_APP=app.py
$ export FLASK_ENV=development
```

No Windows (cmd), os comandos equivalentes são:

```
> set FLASK_APP=app.py
> set FLASK_ENV=development
```

A variável `FLASK_APP` define qual arquivo contém a aplicação. Já `FLASK_ENV` especifica o ambiente de execução — neste caso, `development`, o que habilita o modo de depuração (debug) e recarregamento automático por padrão.

2. Iniciando o servidor

Após configurar as variáveis, inicie o servidor com o seguinte comando:

```
$ flask run
```

↳ deu erro

Você deverá ver uma saída semelhante a esta:

```
* Serving Flask app 'app:create_app("development")'
* Debug mode: on
WARNING: This is a development server. Do not use it in a
    ↪production deployment. Use a production WSGI server ↪
    ↪instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 564-262-181
```

O que aconteceu aqui? O comando `flask run` procurará uma instância do aplicativo Flask no módulo referenciado pela variável de ambiente `FLASK_APP`, que neste caso é `app.py`. O comando configura um servidor web que é configurado para encaminhar solicitações para este aplicativo.

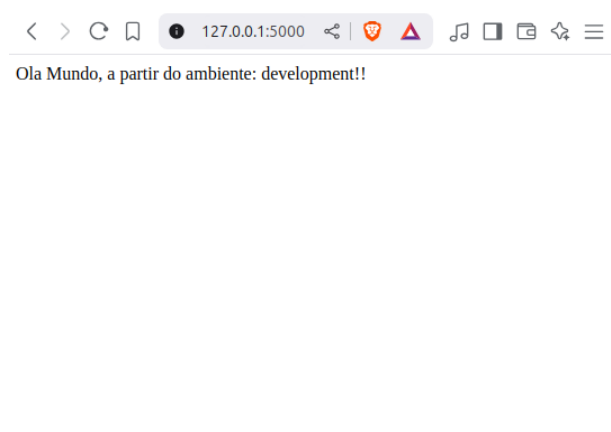
Isso indica que o servidor está rodando localmente na porta 5000. No navegador, acesse:

- `http://127.0.0.1:5000/`
- `http://localhost:5000/`

Ambas as URLs devem exibir a mensagem definida na função `index()`, confirmando que a aplicação está funcionando corretamente.

Após a inicialização do servidor, ele aguardará as conexões do cliente. A saída do `flask run` indica que o servidor está sendo executado no endereço **IP 127.0.0.1**, que é sempre o endereço do seu próprio computador. Este endereço é tão comum que também tem um nome mais simples que você pode ter visto antes: **localhost**. Os servidores de rede escutam conexões em um número de porta específico. Os aplicativos implantados em servidores web de produção geralmente escutam na porta 443, ou às vezes 80 se não implementarem criptografia, mas o acesso a essas portas requer direitos de administração. Como este aplicativo está sendo executado em um ambiente de desenvolvimento, o Flask usa a porta 5000. Agora abra seu

navegador da web e insira a seguinte URL no campo de endereço:



Você vê os mapeamentos de rotas do aplicativo em ação? A primeira URL mapeia para /, enquanto a segunda mapeia para /index. Ambas as rotas são associadas à única função de visualização no aplicativo, então elas produzem a mesma saída, que é a string que a função retorna. Se você digitar qualquer outra URL, obterá um erro, pois apenas essas duas URLs são reconhecidas pelo aplicativo.

Se tudo deu certo até aqui, parabéns! Concluímos nosso primeiro grande passo para nos tornarmos um desenvolvedor web!

3. Verificando o ambiente em tempo de execução

Se tudo estiver configurado corretamente, a página exibida mostrará algo como:

Olá, mundo! Ambiente atual: development.

Para simular outros ambientes (como production), altere a variável `FLASK_ENV` e reinicie o servidor:



```
$ export FLASK_ENV=production
$ flask run
```

Configuração do PowerShell no Windows

Para executar corretamente o `flask run` com suporte ao modo de depuração no PowerShell, é necessário definir as variáveis de ambiente com a sintaxe adequada.

Passos no PowerShell

- 
- Abra o PowerShell na pasta do projeto.
 - Defina as variáveis na mesma sessão (mesma janela do terminal):

```
$env:FLASK_APP = "app.py"  
$env:FLASK_ENV = "development"  
$env:FLASK_DEBUG = "1"
```

- Verifique se foram definidas corretamente:

```
echo $env:FLASK_APP  
echo $env:FLASK_ENV  
echo $env:FLASK_DEBUG
```

Os valores esperados são: `app.py`, `development` e `1`, respectivamente.

- Por fim, execute a aplicação:

```
flask run
```

Explicação

- `FLASK_APP` especifica o ponto de entrada da aplicação.
- `FLASK_ENV=development` ativa o ambiente de desenvolvimento, utilizado na função `create_app()` para aplicar a configuração correta.

- `FLASK_DEBUG=1` ativa explicitamente o modo de depuração, necessário a partir das versões mais recentes do Flask, pois `FLASK_ENV` sozinho já não é suficiente para isso.

Personalizando a URL

Se você prestar atenção em como algumas URLs de serviços que você usa diariamente são formadas, notará que muitas têm seções variáveis. Por exemplo, a URL da sua página de perfil do Facebook tem o formato `https://www.facebook.com/<seu-nome>`, que inclui seu nome de usuário, tornando-o diferente para cada usuário. O Flask suporta esses tipos de URLs usando uma sintaxe especial no decorador `main.route`.

A rota definida no arquivo `app/main/routes.py` permite que a aplicação Flask exiba uma mensagem personalizada com base no nome do usuário incluído diretamente na URL.

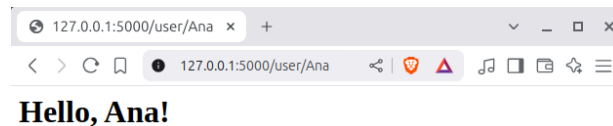
O exemplo a seguir define uma rota que possui um componente dinâmico: **Código:**
`app/main/routes.py`

```
1 from flask import current_app
2 from . import main
3
4 @main.route('/user/<name>')
5 def user(name):
6     return '<h1>Hello, {}!</h1>'.format(name)
```

A rota `/user/<name>` define um parâmetro dinâmico na URL chamado `name`. Quando um usuário acessa, por exemplo, `/user/Ana`, o Flask extrai “Ana” como argumento da função `user(name)` e a insere na resposta HTML. O uso de `<name>` na definição da rota diz ao Flask: “espere um valor aqui e repasse-o à função como argumento”.

A parte da URL da rota entre colchetes angulares é a parte dinâmica. Quaisquer URLs que correspondam às partes estáticas serão mapeadas para esta rota e, quando a função de visualização for invocada, o componente dinâmico será passado como argumento.

No exemplo anterior, o argumento `name` é usado para gerar uma resposta que inclui uma saudação personalizada. Os componentes dinâmicos em rotas são strings por padrão, mas também podem ser de diferentes tipos. Por exemplo, a rota `/user/<int:id>` corresponderia



apenas a URLs que têm um inteiro no segmento dinâmico id, como `/user/123`. O Flask suporta os tipos `string`, `int`, `float` e `path` para rotas. O tipo `path` é um tipo de `string` especial que pode incluir barras, diferentemente do tipo `string`.

Modo Debug

Aplicações Flask podem ser executadas opcionalmente no modo de depuração. Neste modo, dois módulos muito convenientes do servidor de desenvolvimento, chamados `reloader` e `depurador`, estão habilitados por padrão.

Quando o `reloader` está habilitado, o Flask monitora todos os arquivos de código-fonte do seu projeto e reinicia automaticamente o servidor quando qualquer um dos arquivos é modificado. Ter um servidor rodando com o `reloader` habilitado é extremamente útil durante o desenvolvimento, porque toda vez que você modifica e salva um arquivo-fonte, o servidor reinicia automaticamente e detecta a alteração.

O `depurador` é uma ferramenta web que aparece no seu navegador quando o seu aplicativo gera uma exceção não tratada. A janela do navegador web se transforma em um rastreamento de pilha interativo que permite inspecionar o código-fonte e avaliar expressões em qualquer lugar na pilha de chamadas.

NOTA

Nunca habilite o modo de depuração em um servidor de produção. O depurador, em particular, permite que o cliente solicite a execução remota de código, o que

torna seu servidor de produção vulnerável a ataques. Como uma simples medida de proteção, o depurador precisa ser ativado com um PIN, impresso no console pelo comando `flask run`.

O Flask foi projetado para ser estendido. Ele intencionalmente se mantém fora de áreas de funcionalidades importantes, como banco de dados e autenticação de usuários, dando a você a liberdade de selecionar os pacotes que melhor se adaptam à sua aplicação ou de escrever os seus próprios, se desejar.

Uma grande variedade de extensões do Flask para os mais diversos propósitos foi criada pela comunidade e, se isso não for suficiente, qualquer pacote ou biblioteca padrão do Python também pode ser usado.

Execução do Projeto com Ambientes `.env.dev` e `.env.prod`

No desenvolvimento de aplicações Flask, separar configurações por ambiente - como desenvolvimento e produção - é uma prática essencial para garantir segurança, organização e controle. Neste projeto, utilizamos arquivos `.env.dev` e `.env.prod` para armazenar variáveis de ambiente específicas de cada contexto, e empregamos o pacote `invoke` como uma interface de automação para executar tarefas com praticidade e clareza.

Os arquivos `.env.dev` e `.env.prod` são arquivos de texto simples contendo variáveis de ambiente. Eles são lidos automaticamente por bibliotecas como `python-dotenv`, que carrega seu conteúdo para o ambiente de execução Python, conforme apresentamos a seguir.

Código: `.env.dev`

```
1 FLASK_APP = app:create_app("development")
2 FLASK_ENV = development
3 FLASK_DEBUG = 1
```

e Código: `.env.prod`

```
1 FLASK_APP=app:create_app("production")
2 FLASK_ENV=production
3 FLASK_DEBUG=0
```

Esses arquivos não devem ser versionados (adicione ao `.gitignore`), pois podem conter segredos e configurações específicas da máquina local ou servidor.

O pacote `invoke` permite definir tarefas em um arquivo `tasks.py`, de forma similar ao `make`, mas com a flexibilidade do `Python`.

Código: `tasks.py`

```
1 from invoke import task
2 from dotenv import load_dotenv
3
4 @task
5 def run(c, environment="dev"):
6     """
7     Run Flask application based on the specified environment (dev or prod).
8     """
9     # Carregar o arquivo .env correspondente
10    if environment == "prod":
11        load_dotenv(".env.prod")
12    else:
13        load_dotenv(".env.dev")
14
15    # Execute o Flask
16    # invoke run --environment prod
17    c.run("flask run")
```

Com esse `tasks.py`, você pode iniciar a aplicação para diferentes ambientes com comandos simples. Para executar em desenvolvimento:

```
1 invoke run --env=dev
```

Para executar em produção:

```
1 invoke run --env=prod
```

O comando `invoke` chama a função `run`, e o argumento `--env=...` seleciona qual arquivo `.env` será carregado. O conteúdo do `.env` define as variáveis que o `Flask` usará para configurar o ambiente, como `FLASK_APP`, `FLASK_ENV`, `FLASK_DEBUG`, e outros.

Vantagens dessa abordagem:

- **Separação clara de ambientes:** mantêm-se configurações sensíveis fora do código-fonte.
- **Automação padronizada:** o `invoke` oferece uma interface clara para tarefas comuns como `run`, `test`, `build`, etc.
- **Flexibilidade para deploys:** em produção, basta ter um `.env.prod` ajustado ao servidor e executar o mesmo comando.
- **Boas práticas modernas:** evita o uso de `export VAR=...` no `shell`, centralizando tudo em arquivos bem definidos.

Portanto, usar arquivos `.env` combinados com `invoke` proporciona uma maneira elegante, segura e controlada de executar aplicações Flask em diferentes ambientes. Isso favorece a escalabilidade do projeto e melhora a experiência de desenvolvimento e manutenção.

NOTA

Em projetos maiores, essa estrutura pode ser expandida para incluir ambientes de teste, homologação ou staging, bastando adicionar novos arquivos `.env` e adaptar o `tasks.py`.

Notas finais

- A linha `app.debug = app.config.get("DEBUG", False)` no código garante que o modo de depuração da aplicação estará sincronizado com as configurações do ambiente selecionado.
- Caso prefira executar diretamente com `python app.py`, o modo debug será ativado conforme a configuração carregada ou passado como argumento para `app.run(debug=True)`.
- Para evitar configurar as variáveis manualmente sempre que iniciar o terminal, você pode usar um arquivo `.env` em conjunto com a biblioteca `python-dotenv`. Assim, as variáveis são carregadas automaticamente na inicialização do ambiente.

Lista de Exercícios

2.1 Qual das alternativas abaixo **melhor descreve a função de um ambiente virtual em Python?** 

- (a) Permitir executar o Flask sem o uso do terminal.
- (b) Aumentar o desempenho do interpretador Python. 
- (c) Executar múltiplas aplicações Flask simultaneamente. 
- (d) Proteger o código contra falhas de segurança. 
- (e) Isolar dependências e evitar conflitos entre projetos. ☒

2.2 Qual comando cria um ambiente virtual chamado venv?

- (a) flask create venv 
- (b) python3 venv create
- (c) python3 -m venv venv ☒ 
- (d) pip install venv
- (e) python -m flask new-env

2.3 No padrão Application Factory, qual é o papel da função create_app()? 

- (a) Registrar as rotas diretamente em app.py. 
- (b) Criar e retornar uma instância configurada do Flask. ☒
- (c) Instalar o Flask e suas dependências. 
- (d) Criar automaticamente um ambiente virtual. 
- (e) Executar o servidor Flask em produção.

2.4 Qual é o nome da variável de ambiente usada para indicar o ponto de entrada da aplicação Flask?

(a) FLASK_APP ☒

(b) FLASK_DEBUG

(c) FLASK_RUN

(d) FLASK_ENTRY

(e) APP_FLASK

2.5 O que são *blueprints* no contexto do Flask?

(a) Um tipo especial de template HTML.

(b) Um sistema de cache para acelerar o Flask.

(c) Um comando para automatizar a instalação do Flask.

(d) Um modo de debug para aplicações complexas.

(e) Um mecanismo para separar funcionalidades em módulos reutilizáveis. ☒

2.6 Qual biblioteca abaixo **não** é uma dependência principal do Flask?

(a) Werkzeug

(b) Jinja2

(c) Click

(d) SQLAlchemy ☒

(e) Nenhuma das anteriores

2.7 Em qual arquivo deve ser definida a função `create_app()` segundo a estrutura modular do projeto?

(a) `app/__init__.py` ☒

(b) `config.py`

(c) `app.py`

(d) `main/routes.py`

(e) `venv/__init__.py`

2.8 Qual das alternativas corresponde a uma rota registrada corretamente em um blueprint chamado main?

- (a) `@app.route('/') `
- (b) `@route.main('/') `
- (c) `@main.route('/') ☒`
- (d) `@flask.route('/') `
- (e) `@Blueprint.route('/') `

2.9 Para ativar o ambiente virtual venv no Linux ou macOS, qual comando deve ser utilizado?

- (a) `activate venv`
- (b) `venv/activate.sh`
- (c) `run venv`
- (d) `bash activate venv`
- (e) `source venv/bin/activate ☒`

2.10 O que a seguinte linha de código faz no `routes.py`?


`ambiente = current_app.config.get("CONFIG_NAME", "desconhecido")`

- (a) Cria um novo ambiente virtual.
- (b) Recupera o nome do ambiente de configuração ativo. ☒
- (c) Define o nome do blueprint.
- (d) Obtém a variável de ambiente do sistema operacional.
- (e) Importa o nome do pacote principal.

Referências Bibliográficas

- [1] GRINBERG, M. *Flask Web Development: Developing Web Applications with Python*. 2. ed. Sebastopol: O'Reilly Media, 2018. ISBN 9781491991732.
- [2] BEAZLEY, D. *Python Distilled*. Boston: Addison-Wesley Professional, 2021. Livro que apresenta uma visão concisa e moderna da linguagem Python. ISBN 9780134173276.